

Where The Project Stands & What To Do Next

A full done/not-done checklist, what three training runs have shown, a problem in the Indonesian evaluation dataset that must be fixed before it can be used, and the next experiments worth running.

336 tests passing

Calibration: PASS

Training gate: not met yet

Dataset: needs a fix

Project: TELITI — job-ad integrity scoring

Repo: github.com/Gigvo/TELITI

Latest run: mDistilBERT, 512 tokens, 3 epochs, RTX 5050, 26 min

Related: [docs/GPU_HANDOFF.md](#), [docs/INDONESIAN_DATASET_SPEC.md](#), [STATUS.md](#)

1	Checklist — what is done, what is not	Page 2
2	What the training runs show	Page 4
3	The Indonesian dataset — fix this first	Page 6
4	What to do next	Page 8
5	Quick reference	Page 10

1 — Checklist

Roughly 65% complete. Everything around the model — API, rule engine, scoring, frontend, deployment, evaluation harness — is built and tested. What remains is the model itself clearing its bar, and the evaluation dataset being usable.

SETUP & DATA

✓	0.1	Python environment, dependencies installed	done
✓	0.2	GPU verified on RTX 5050 (cu128 wheels, real matmul)	done
✓	0.3	EMSCAD dataset downloaded and validated	17,880 rows
✓	0.4	Repository structure, git, GitHub remote	done
✓	1.2	Data preparation — 4-way stratified split	12.5k / 1.8k ×3
✓	1.2b	Derivability measured (EMSCAD cannot teach rule weights)	done

CORE SYSTEM

✓	1.1	API contract frozen (<code>api/schemas.py</code>)	done
✓	1.3	TF-IDF baselines trained	PR-AUC 0.8769
✓	1.4	Rule engine + email / contact rules (5 features)	done
✓	2.4	Remaining rules — salary, qualification, risk phrases (4 features)	all 9 filled
✓	–	Locale layer (English + Indonesian, drop-in)	done
✓	3.1a	Scoring layer — bounded rule weights, fusion	done
✓	1.6	React frontend — score gauge, evidence highlighting	19 tests
✓	4.3	Demo hardening — sanitisation, latency 794→86 ms	done
✓	–	Docker deployment config	not build-tested

MODEL

🟡	2.1	Fine-tune transformer — 3 runs, none past the gate yet	best 0.8763
✓	2.3	Probability calibration (Platt scaling)	ECE 0.0038
□	2.5	Wire the real model into the API	needs 2.1

EVALUATION

🟡	1.5	Indonesian holdout — 200 items collected, ~85 need full text	see §3
□	2.2	Cross-language probe — the headline number	needs 1.5
□	3.1b	Validate rule weights on the holdout	needs 1.5
□	3.2	Risk thresholds from a precision target	needs 2.1

SERVING & DELIVERY

☐	3.3	ONNX export + INT8 quantisation	needs 2.1
☐	3.4	Sentence-level explainability (occlusion)	needs 2.1
☐	4.1	Full pipeline wired end to end	needs 2.5
☐	4.2	Final evaluation report (test split, touched once)	needs 2.1 + 1.5
☐	4.4	Tag <code>v0.1-mvp</code> — freeze point	last step

WRITE-UP

☒	—	Paper revisions drafted (5 changes, Indonesian text ready)	not applied
----------------------------------	---	--	-------------

✓ done · ☒ partial / in progress · ☐ not started

The two things actually blocking progress

BLOCKER	BLOCKS	WHO
Dataset: ~85 holdout entries contain index rows, not full ad text	2.2, 3.1b, 4.2 — every headline number	Data work, no GPU
Model: transformer has not beaten the TF-IDF baseline	2.5, 3.2, 3.3, 3.4, 4.1	GPU experiments

These are independent — different people can work them in parallel.

2 — What the training runs show

All results so far

RUN	MAX_LEN	EPOCHS	PR-AUC	VS BASELINE	TIME
TF-IDF + LinearSVC (CPU, no GPU needed)	—	—	0.8769	—	9 sec
mDistilBERT, run 2 (GPU)	256	3	0.8763	-0.0006	14 min
mDistilBERT, run 3 (GPU) — latest	512	3	0.8595	-0.0174	26 min

A first CPU run was abandoned partway through when the GPU machine became available; it is not comparable and is excluded.

The diagnosis: undertrained, not overfitting

The evaluation history from the latest run tells the story clearly:

STEP	PR-AUC	CHANGE	PRECISION	RECALL	FALSE POSITIVES
400	0.5724	—	0.505	0.628	53
800	0.7401	+0.168	0.815	0.616	12
1200	0.7967	+0.057	0.894	0.686	7
1600	0.8395	+0.043	0.938	0.709	4
2000	0.8562	+0.017	0.967	0.686	2
2349	0.8595	+0.003	0.899	0.721	7

PR-AUC climbed monotonically to the final step — it never peaked and declined. That is the signature of a model that ran out of training budget, not one that overfit. An overfitting model shows PR-AUC rising then falling while training loss keeps dropping. This one was still improving when training stopped.

Why 512 tokens scored worse than 256

Counterintuitive, but it is a scheduling artefact rather than a finding about context length.

Both runs used the same 2,349 optimizer steps. Step count is `dataset_size ÷ batch_size × epochs` — it does not change when you lengthen the sequence. So the 512-token run had exactly as many gradient updates to learn a *harder* problem: twice the tokens per example, more positions to attend over. It simply did not get there.

There is a second effect that makes this worth rerunning rather than extrapolating from the curve. The learning rate follows a **linear decay to zero** across the planned steps. By step 2349 the learning rate was essentially nil, so the final checkpoints were barely learning at all. Running more epochs does not merely append more of that flat tail — it *stretches the entire schedule*, giving the model far more time at a productive learning rate.

One point in the transformer's favour

At **step 2000: precision 0.967 with only 2 false positives** out of 1,702 legitimate ads. The TF-IDF baseline's best is 0.930 precision with 5.

PR-AUC averages performance across *all* thresholds, but this product does not operate at all thresholds. Concept paper §3.6 states that false positives against real companies are the expensive error, and step 3.2 selects the operating point from a precision target. **The transformer appears able to reach a higher precision ceiling than TF-IDF** — which may matter more for the product than the PR-AUC summary suggests.

This is one checkpoint and the comparison needs a consistent threshold rule, so it is not yet a conclusion. But it is a reason to measure properly at step 3.2 rather than discarding the transformer on PR-AUC alone.

Calibration — still a clean pass

METRIC	BEFORE	AFTER	MEANING
ECE	0.0113	0.0038	Gate was ≤ 0.05 — roughly 13× under it
Brier	0.0148	0.0138	Improved
PR-AUC	0.8839	0.8839	Unchanged — correct, Platt scaling is monotone and cannot reorder

The Integrity Score now means something concrete: when the system outputs 0.80, it is right roughly 80% of the time.

3 — The Indonesian dataset: fix this first

Correction to revision 1 of this brief. An earlier version stated the Indonesian evaluation dataset "does not exist." That was wrong — written after checking only one of the two machines. It does exist, at `eval/indonesian_holdout.jsonl`, with 200 items and a clean validator run.

What's there

```
items      : 200 / 200 target
labels     : 72 scam, 128 legitimate (36.0% scam)
channels   : {'job_board': 123, 'whatsapp': 44, 'other': 33}
source types : {'job_board': 110, 'community_report': 72,
                'company_official': 10, 'glints': 5, 'media': 3}
looks good.
```

Target size met, and the 36% scam rate is almost exactly the 35% the spec asked for — deliberately legitimate-leaning so false positives, the expensive error, are what gets surfaced.

The problem: 43% of items are not job ads

86 of the 200 items are under 200 characters, and 85 of those are labelled legitimate. They look like this:

```
Driver • Cititrans • Kompetitif • SMA/SMK • 1-2 thn • Luar Jabodetabek
Staff Gudang • Roschel • Kompetitif • SMA/SMK • 0-2 thn • Jakarta Barat
Kasir • Andifa Vet Clinic • Kompetitif • SMA/SMK • 0-2 thn • Jakarta Selatan
Helper Mekanik • PT. Anugrah Vindo Abadi • Kompetitif • SMK • 1-2 thn • Bekasi
```

Those are **job-board index rows** — the one-line summary from a search results page, not the advertisement itself. Every scam item, by contrast, is a full forwarded WhatsApp message with a median length of **767 characters**.

CLASS	MEDIAN LENGTH	MIN	MAX
Legitimate	125 chars	70	1,289
Scam	767 chars	185	2,478

Why this invalidates the evaluation

Text length ALONE as a scam detector on this holdout:

PR-AUC 0.8847

ROC-AUC 0.9355

Best single threshold rule: "text >= 500 chars => scam"

F1 0.8082 precision 0.7973 recall 0.8194 (TP 59, FP 15, FN 13)

Counting characters scores PR-AUC 0.8847 on this dataset — higher than either trained transformer managed on EMSCAD. Any model evaluated here will post an impressive number that is substantially just

measuring "is this text long?", and the score alone gives no way to tell the difference.

It also does not match the product's real input. A user pastes a job ad they received. Nobody will paste a 70-character index row into TELITI, so those items measure performance on input that never occurs in practice.

The good news — close, not broken

114 of the 200 items are already exactly right. The Glints entries and the longer `job_board` ones are proper full postings:

```
[id-holdout-0088] Pharos Indonesia • Medical Representative
Mempromosikan produk farmasi ke dokter, apotek, rumah sakit. Kerja
lapangan, target-oriented.
- Pria/Wanita, pendidikan minimal D3/S1
- Komunikatif dan berorientasi pada target
- Siap bekerja di lapangan ...
```

The fix: open each short entry's `source_url`, click through to the actual posting, and copy the **full advertisement body** instead of the summary line. Same items, same labels, same sources — only `text` changes. Roughly 85 entries.

Two smaller issues

Source perfectly predicts label

SOURCE_TYPE	LEGITIMATE	SCAM
<code>community_report</code>	0	72
<code>job_board</code>	110	0
<code>company_official</code>	10	0
<code>glints</code>	5	0
<code>media</code>	3	0

Zero overlap. `source_type` is not fed to the model, so this is not leakage in the strict sense — but the two classes differ systematically in origin, and the length gap is one symptom of that. Worth remembering when interpreting whatever score comes out.

Zero Telegram items

The `contact_messaging_only` rule deliberately weights Telegram at **0.75** versus WhatsApp at **0.35**, because in Indonesia WhatsApp is ordinary business communication while Telegram is not. That asymmetry is one of the more considered Indonesian-context decisions in the rule layer, and nothing in the holdout currently tests it. A handful of Telegram-channel scam items would fix that.

4 — What to do next

DO FIRST — GPU, ~28 MIN

Experiment 3: more epochs at 256 tokens

The cheapest test of the undertraining diagnosis, starting from the better position — 256 tokens already reached 0.8763 after 3 epochs.

```
python ml/train_transformer.py --no-freeze --epochs 6 --max-length 256
```

Doubling the budget also stretches the learning-rate schedule, so the model spends far more time at a productive rate rather than decaying to zero early.

Compare against 0.8769.

DO FIRST — DATA WORK, NO GPU

Fix the ~85 short dataset entries

Replace index-row text with the full advertisement body from each entry's `source_url`. Labels, ids and sources stay the same.

To list exactly which entries need it:

```
python -c "
import json
rows=[json.loads(l) for l in open('eval/indonesian_holdout.jsonl',encoding='utf-8') if l.strip()]
for r in rows:
    if len(r['text']) < 200:
        print(r['id'], len(r['text']), r['source_url'])
"
```

Re-validate when done: `python ml/validate_eval_set.py`

Independent of the GPU work — both can run in parallel.

THEN — GPU, ~52 MIN, ONLY IF EXPERIMENT 3 FALLS SHORT

Experiment 4: more epochs at 512 tokens

```
python ml/train_transformer.py --no-freeze --epochs 6 --max-length 512
```

Gives the longer-context model the budget it clearly needed. Add `--batch-size 8` if CUDA runs out of memory.

Set a stopping rule now, before running these. If neither experiment clears 0.8769, stop. "A fine-tuned multilingual transformer matches but does not beat a TF-IDF baseline on EMSCAD" is a legitimate, reportable

result. Continuing to tune past that point is how a project ends up overfitting to its own validation split by hand — and the number stops meaning anything.

AFTER THE DATASET IS FIXED

Run the cross-language evaluation (step 2.2)

This is the number the whole project is actually about: does a model trained on 2017 English job-board data work on 2026 Indonesian chat-app scams?

Compare against 0.8847, not against zero. That is what text length alone achieves on this holdout. If the model does not clearly exceed it, it has learned nothing beyond document length — and that must be reported, not smoothed over.

WHENEVER A RUN FINISHES

Send back the artifacts

PATH	WHY
artifacts/scam_model/training_summary.json	Eval history at every checkpoint — the file that made this diagnosis possible
artifacts/scam_model/	Trained weights
artifacts/calibrator.json	Platt parameters
eval/calibration_report.md	Reliability tables

Simplest: zip the whole `artifacts/` directory. These are gitignored (too large to push), so they move by Drive / Discord / USB.

5 — Quick reference

Commands

PURPOSE	COMMAND
Experiment 3 (recommended next)	<code>python ml/train_transformer.py --no-freeze --epochs 6 --max-length 256</code>
Experiment 4 (fallback)	<code>python ml/train_transformer.py --no-freeze --epochs 6 --max-length 512</code>
Resume after interruption	<code>python ml/train_transformer.py --resume</code>
Check progress mid-run	<code>python ml/training_status.py --pid <PID></code>
Calibrate after training	<code>python ml/calibrate.py</code>
Validate the dataset	<code>python ml/validate_eval_set.py</code>
Run the test suite	<code>python -m pytest -q</code>

Gate status

GATE	TARGET	STATUS	INTERPRETATION
2.1 — training	val PR-AUC ≥ 0.88	NOT MET (best 0.8763)	Undertrained, still climbing. Two experiments left, then accept the finding.
2.3 — calibration	ECE ≤ 0.05 and improved	PASS (0.0038)	Clean success. The Integrity Score is meaningful.
1.5 — dataset	~200 realistic items	PARTIAL	200 collected and balanced; ~85 need full ad text instead of index rows.
1.3 — baseline	PR-AUC ≥ 0.80	PASS (0.8769)	Strong. Remains a viable no-GPU fallback if the transformer never wins.

Troubleshooting

SYMPTOM	FIX
CUDA out of memory at <code>--max-length 512</code>	Add <code>--batch-size 8</code>
<code>no kernel image is available</code>	Wrong PyTorch wheel — reinstall from the <code>cu128</code> index
Training loss becomes <code>nan</code>	Lower <code>--lr</code> (try <code>1e-5</code>)
Run interrupted	<code>--resume</code> picks up from the last checkpoint

What this changes in the concept paper

§3.3 presents the transformer as the core AI contribution. There is now measured evidence from three runs that, on EMSCAD, it **matches but does not beat** a classical TF-IDF baseline. That needs stating plainly.

This is not a weakness in the write-up — it is a stronger methods section than an unsupported claim. And if the transformer does outperform on a properly constructed Indonesian holdout — plausible, since TF-IDF learned untransferable English tokens like `subsea` and `below link` — the contrast becomes the most interesting result in the paper rather than an awkward footnote.

Exact wording is drafted in `docs/PAPER_REVISIONS.md` and will be finalised once the experiments settle on a number.